

Amazon Web Services

Complete Learning Guide

Beginner-Friendly · In-Depth · Practical

A comprehensive reference covering all major AWS Management and Governance services — from infrastructure-as-code and change tracking to observability, compliance, and well-architected reviews — with real-world use cases, architecture diagrams, comparison tables, and hands-on code examples.

Services Covered

#	Service	Core Purpose
1	AWS CloudFormation	Infrastructure as Code — Provision Everything
2	AWS CloudTrail	API Audit Logging — Who Did What, When, Where
3	Amazon CloudWatch	Unified Observability — Metrics, Logs, Alarms, Dashboards
4	AWS Config	Continuous Configuration Compliance & Change History
5	Amazon Managed Grafana	Managed Visualization for Metrics & Operational Data
6	AWS Systems Manager	Unified Operations Hub — Manage AWS at Scale
7	AWS Well-Architected Tool	Evaluate & Improve Cloud Architecture Quality

Version 1.0 · AWS Certified Training Material

Table of Contents

1. AWS CloudFormation — *Stacks · Templates · Drift Detection · StackSets · CDK*

- 1.1 Introduction & Analogy
- 1.2 Core Concepts
- 1.3 Architecture & Working
- 1.4 Features
- 1.5 Real-World Use Cases
- 1.6 Practical Example
- 1.7 Integrations & Comparison

2. AWS CloudTrail — *Events · Trails · Insights · Athena Integration · Lake*

- 2.1–2.7 Full Coverage

3. Amazon CloudWatch — *Metrics · Logs · Alarms · Dashboards · Synthetics · X-Ray*

- 3.1–3.7 Full Coverage

4. AWS Config — *Rules · Conformance Packs · Remediation · Aggregators*

- 4.1–4.7 Full Coverage

5. Amazon Managed Grafana — *Workspaces · Data Sources · Dashboards · SAML/SSO*

- 5.1–5.7 Full Coverage

6. AWS Systems Manager — *Parameter Store · Session Manager · Patch Manager · Automation*

- 6.1–6.7 Full Coverage

7. AWS Well-Architected Tool — *Pillars · Lenses · Milestones · Improvement Plans*

- 7.1–7.7 Full Coverage

Appendix: Master Comparison Table — *All 7 Services Side-by-Side*

AWS CloudFormation

Infrastructure as Code — Model, Provision & Manage AWS Resources Declaratively

1.1 Simple Introduction

What is AWS CloudFormation?

AWS CloudFormation is a service that lets you model, provision, and manage your entire AWS infrastructure using code — declarative templates written in YAML or JSON. Instead of clicking through the AWS Console to create an EC2 instance, VPC, RDS database, and IAM role separately (and then trying to remember what you did six months later), you write a single CloudFormation **template** that describes all those resources, their configuration, and their relationships. CloudFormation then creates, updates, and deletes them in the correct order, automatically.

Why Does It Exist?

Manual infrastructure provisioning is slow, error-prone, and impossible to reproduce exactly. If you build a production environment by clicking through the console, how do you build an identical staging environment? How do you track what changed last Tuesday? CloudFormation answers these questions by treating infrastructure exactly like application code — version-controlled, repeatable, peer-reviewed, and automated.

■ **CloudFormation is like an architect's blueprint for a building.** Instead of verbally instructing each tradesperson (plumber, electrician, carpenter) separately — and hoping they coordinate — you provide a single master blueprint. The construction manager (CloudFormation) reads the blueprint, knows the dependency order (foundation before walls, walls before roof), and coordinates all trades simultaneously where possible. If you want an identical building next door, just hand the same blueprint to a new construction manager.

Business Problems it Solves

Problem	How CloudFormation Solves It
Manual click-ops is slow & error-prone	Declare infrastructure in code; provision 100 resources in minutes, identically every time
Can't reproduce environments (dev vs prod)	Same template → deploy to any account/region with parameter overrides for env differences
No audit trail for infrastructure changes	Template in Git = full change history. CloudFormation tracks stack events and drift
Deleting resources is risky and tedious	Delete the stack → CloudFormation removes all resources in correct reverse-dependency order
Multi-account/multi-region rollout is hard	StackSets deploy one template to hundreds of accounts/regions simultaneously
Infrastructure and app code are disconnected	AWS CDK lets developers write infrastructure in Python/TypeScript that synthesizes to CloudFormation

1.2 Core Concepts & Terminology

Term	Definition
Template	A YAML or JSON document describing the AWS resources to create. The blueprint. Stored in S3 or local.
Stack	A deployed instance of a template. All resources in a stack are managed together as a unit.
Resource	An AWS resource declared in the template: EC2 instance, S3 bucket, Lambda function, VPC, etc.
Parameter	Input variable passed to a template at deploy time — allows one template to serve multiple environments (dev/prod).
Output	Value exported from a stack (e.g., VPC ID, Load Balancer DNS) — importable by other stacks.
Mapping	Static lookup table in the template — map region to AMI ID, or environment to instance size.
Condition	Boolean logic in template — create a resource only if a condition is true (e.g., only in prod).
Intrinsic Function	Built-in functions: Ref (reference a resource/param), Fn::GetAtt (get attribute), Fn::Sub (string interpolation), Fn::If, Fn::Select, etc.
Change Set	Preview of what CloudFormation WILL change before you execute — shows adds, modifications, deletions.
Drift Detection	Compares actual resource state against template definition — flags manual changes made outside CloudFormation.
Stack Policy	JSON policy that protects specific resources from accidental update or deletion during stack updates.
StackSets	Deploy a single CloudFormation template across multiple AWS accounts and regions simultaneously.
Nested Stack	A stack that is a resource in another stack — breaks large templates into reusable modules.
CloudFormation Registry	Extend CloudFormation to manage third-party resources (Datadog monitors, GitHub repos, etc.) via resource providers.
AWS CDK	AWS Cloud Development Kit — write infrastructure in Python, TypeScript, Java, C# that synthesizes to CloudFormation templates.

1.3 Architecture & Working

DEVELOPER / CI-CD PIPELINE

Writes / updates CloudFormation Template (YAML/JSON) → commits to Git

Uploads template to Amazon S3 (if >51KB)

↓ CreateStack / UpdateStack API call

CLOUDFORMATION SERVICE (fully managed by AWS)

1. Template validation – checks syntax and resource types
2. Dependency graph construction – determines resource creation order
3. Change Set generation – what will be created/modified/deleted
4. Execution – calls AWS APIs to provision resources in parallel where possible

Resources with dependencies wait for their dependencies to complete

5. Rollback – if ANY resource fails, all changes rolled back automatically

6. Stack Events – every action logged with timestamp, status, reason

↓ Stack Status: CREATE_COMPLETE / UPDATE_COMPLETE / ROLLBACK_COMPLETE

PROVISIONED RESOURCES (EC2, VPC, RDS, Lambda, IAM, S3...)

All tagged with `aws:cloudformation:stack-name` for tracking

↓

OUTPUTS exported and available for import by other stacks (cross-stack references)

Template Anatomy

```
AWSTemplateFormatVersion: '2010-09-09'
```

```
Description: 'Web Application Stack'
```

```
Parameters:
```

```
  Environment:
```

```
    Type: String
```

```
    AllowedValues: [dev, staging, prod]
```

```
Mappings:
```

```
  InstanceSizeMap:
```

```
    dev:    { InstanceType: t3.micro }
```

```
    staging: { InstanceType: t3.small }
```

```
    prod:   { InstanceType: t3.large }
```

```
Conditions:
```

```
  IsProd: !Equals [!Ref Environment, prod]
```

```
Resources:
```

```
  WebServer:
```

```
    Type: AWS::EC2::Instance
```

```
    Properties:
```

```
      InstanceType: !FindInMap [InstanceSizeMap, !Ref Environment, InstanceType]
```

```
      # ... more properties
```

```
  ProdOnlyAlarm:      # Only created in prod
```

```
    Type: AWS::CloudWatch::Alarm
```

```
    Condition: IsProd
```

```
    Properties: { ... }
```

```
Outputs:
```

```
  WebServerIP:
```

```
    Value: !GetAtt WebServer.PublicIp
```

```
    Export:
```

```
      Name: !Sub '${AWS::StackName}-WebIP'
```

1.4 Major Features

Feature	Description	Business Value
Change Sets	Preview exact changes before executing — see what will be deleted/modified	Prevent accidental production outages
Rollback on Failure	If stack update fails, CloudFormation automatically reverts to last good state	Self-healing infrastructure deployments

Feature	Description	Business Value
Drift Detection	Detect when someone manually changed a resource outside CloudFormation	Enforce IaC discipline; find compliance gaps
StackSets	Deploy to 100+ accounts/regions with one command (Org-wide or account list)	Standardize infrastructure across your entire AWS Organization
Nested Stacks	Reference other stacks as resources — modular, reusable infrastructure components	DRY (Don't Repeat Yourself) for infrastructure
CloudFormation Guard (cfn-guard)	Policy-as-code — validate templates against rules before deployment	Shift-left compliance: catch misconfigs in CI/CD
Resource Import	Import existing (manually-created) resources into a stack without recreating them	Bring legacy infrastructure under IaC control
Modules	Reusable template fragments — share standardized patterns across teams (from CloudFormation Registry)	Enforce org-wide standards (VPC layout, tagging, security)
AWS CDK Integration	Write Python/TypeScript/Java → CDK synthesizes to CloudFormation JSON	Developers use familiar languages; L3 constructs provide sane defaults

1.5 Real-World Use Cases

Industry	Use Case	CloudFormation Pattern
SaaS Product	On-board each new customer with a dedicated isolated stack (VPC, RDS, Lambda, S3) in under 10 minutes	CreateStack via API per customer
Enterprise IT	Standardize VPC, subnet, NAT, route table layout across 500 AWS accounts via StackSets	StackSets + Service Control Policies
DevOps / CI-CD	GitHub Actions workflow: on PR merge → deploy CloudFormation stack → run tests → promote to prod	Change Sets in pipeline
Healthcare / HIPAA	Enforce encryption-at-rest on all S3 buckets via cfn-guard in CI/CD pipeline — block non-compliant templates	cfn-guard + CloudFormation Guard rules
Fintech	Disaster recovery: same template deploys primary (us-east-1) and DR (us-west-2) stacks identically	Multi-region deploy with parameters
Startup	AWS CDK app defines entire infrastructure in Python — developers own infra alongside application code	CDK + CodePipeline

1.6 Practical Example — Three-Tier Web App Stack (YAML)

```

AWSTemplateFormatVersion: '2010-09-09'
Parameters:
  DBPassword:
    Type: String
    NoEcho: true
    # Password not shown in console/logs

```



```
Action: sts:AssumeRole
```

Outputs:

DatabaseEndpoint:

Value: !GetAtt AppDatabase.Endpoint.Address

Export:

Name: !Sub '\${AWS::StackName}-DBEndpoint'

1.7 Integrations & Comparison

AWS Service	Integration with CloudFormation
AWS CDK	CDK synthesizes Python/TypeScript/Java code into CloudFormation templates. CDK deploy calls CloudFormation.
AWS CodePipeline	Pipeline stage: deploy CloudFormation stack/change set as part of CI/CD pipeline.
Amazon S3	Templates >51KB stored in S3. S3 is the recommended template storage location.
AWS IAM	CloudFormation assumes an IAM role (or uses caller's permissions) to create resources.
AWS Config	Config tracks changes to CloudFormation stacks. Config rules can check stack drift.
AWS Service Catalog	Service Catalog products are CloudFormation templates exposed to end users with guardrails.
AWS Systems Manager	Parameter Store values referenced in templates via ssm dynamic references. SSM Automation uses CFN.
Amazon SNS	Stack notifications — CloudFormation publishes stack events to an SNS topic.

CloudFormation vs Terraform vs CDK

Dimension	AWS CloudFormation	Terraform (HashiCorp)	AWS CDK
Type	Declarative IaC (AWS-native)	Declarative IaC (multi-cloud)	Imperative IaC (→ CloudFormation)
Language	YAML / JSON	HCL (HashiCorp Configuration Language)	Python, TypeScript, Java, C#, Go
Cloud Support	AWS only	AWS, Azure, GCP, 1000+ providers	AWS (via CloudFormation)
State Management	Managed by AWS (no state file)	Requires state file (S3 + DynamoDB lock)	Managed by CloudFormation
Rollback	Automatic on failure	Manual (terraform destroy + re-apply)	Automatic (via CloudFormation)
Learning Curve	Medium (YAML verbose)	Medium (HCL clean)	Low for developers (familiar languages)
Best For	AWS-native, enterprise governance	Multi-cloud, existing Terraform teams	Developers who prefer code over YAML

AWS CloudTrail

API Activity Auditing — Complete Governance, Compliance & Operational Visibility

2.1 Simple Introduction

What is AWS CloudTrail?

AWS CloudTrail is a service that continuously records API activity across your entire AWS account — who made a call, what action they took, which resource was affected, when it happened, and from where. Every time anyone (or any service) calls an AWS API — creating an EC2 instance, deleting an S3 bucket, changing an IAM policy, launching a Lambda function — CloudTrail captures that event as a log record and delivers it to Amazon S3. CloudTrail is the foundation of security, compliance, and operational auditing on AWS.

The Core Question CloudTrail Answers

'Who did what to which resource, when, and from where?' This question is critical for: security incident investigation ('who deleted that S3 bucket?'), compliance auditing ('prove that only authorized users accessed patient data'), operational troubleshooting ('which API call caused that Lambda to fail?'), and cost attribution ('which team launched 200 EC2 instances last night?').

■ **CloudTrail is like a building's CCTV system, but for every door and action.** Every time someone enters a room (calls an AWS API), swipes a badge (authenticates), picks up a file (accesses an S3 object), or changes the thermostat (modifies a config), it's recorded on camera. Security teams review footage after an incident, compliance auditors verify that only authorized staff entered restricted areas, and building managers get alerts when someone accesses the server room at 3 AM.

2.2 Core Concepts & Terminology

Term	Definition
Event	A single recorded API activity. Contains: who (userIdentity), what (eventName), which resource (resources), when (eventTime), where (sourceIPAddress), and outcome (errorCode if failed).
Management Event	(Also: Control Plane event) Operations on AWS resources: CreateBucket, RunInstances, DeleteTable, CreateUser. High-level resource management. Enabled by default.
Data Event	(Also: Data Plane event) Operations ON data inside resources: S3 GetObject/PutObject, Lambda Invoke, DynamoDB GetItem. High-volume. Optional (extra cost).
Insights Event	Unusual API activity detected by CloudTrail ML: sudden spike in API error rate, unusual IAM action volume, unexpected resource creation burst.
Trail	Configuration that specifies which events to capture and where to deliver them (S3 bucket, optionally CloudWatch Logs and EventBridge).
Multi-Region Trail	A trail that captures events from ALL regions in one S3 bucket. Recommended — catches events in regions you don't normally use.

Term	Definition
Organization Trail	A trail created in the AWS Organizations management account that automatically covers ALL member accounts.
CloudTrail Lake	Managed event data store for querying CloudTrail history using SQL — no S3 + Athena setup needed.
Event History	Free 90-day searchable history of management events in the console — no trail required.
Log File Integrity Validation	CloudTrail creates a SHA-256 hash digest of each log file — detect if logs were tampered with after delivery.
S3 Log Destination	All trail events delivered to an S3 bucket as gzip-compressed JSON log files (partitioned by account/region/date).

2.3 Architecture & Working

AWS API CALLS (Console ■ CLI ■ SDK ■ AWS Services themselves)

↓ Every API call captured by CloudTrail

CLOUDTRAIL SERVICE

Receives event → enriches with caller identity, source IP, user agent

Validates against Trail configuration (which event types, which regions)

↓ Typically within 15 minutes of API call

DELIVERY DESTINATIONS (all optional, configured per Trail)

- Amazon S3 — gzip JSON log files (long-term archive) ■
- CloudWatch Logs — real-time stream for alarms and search ■
- Amazon EventBridge — real-time event routing to targets ■
- CloudTrail Lake — SQL-queryable event data store (7y max) ■

↓ Analysis layer

QUERY / ALERT LAYER

Amazon Athena — SQL queries on S3 CloudTrail logs

CloudWatch Alarms — alert on specific API patterns

Amazon OpenSearch — index and search trail logs

Amazon GuardDuty — uses CloudTrail as one of its input feeds

AWS Security Hub — correlates CloudTrail findings

CloudTrail Event Record Structure

```
{
  "eventVersion": "1.08",
  "userIdentity": {
    "type": "IAMUser",
    "principalId": "AIDAXXXXXXXXXXXXXXXXXX",
    "arn": "arn:aws:iam::123456789012:user/alice",
    "accountId": "123456789012",
    "userName": "alice"
  },
  "eventTime": "2024-05-15T14:23:11Z",
  "eventSource": "s3.amazonaws.com",
```

```

"eventName": "DeleteBucket",           ← WHAT was done
"sourceIPAddress": "192.168.1.100",    ← FROM where
"requestParameters": {
  "bucketName": "production-data-lake" ← WHICH resource
},
"responseElements": null,
"errorCode": null,                    ← SUCCESS (no error)
"awsRegion": "us-east-1"
}

```

2.4 Major Features

Feature	Description
Event History (Free)	90 days of management events searchable in the console. No trail needed. Filter by resource, user, time, event type.
Multi-Region / Org Trail	Single trail captures all events across all regions and all accounts in an AWS Organization.
Log File Integrity	SHA-256 digest files prove logs haven't been tampered with — required for many compliance frameworks (PCI-DSS, HIPAA).
CloudTrail Insights	ML detects unusual API patterns: sudden IAM role assumption spike, unexpected high-volume Delete* calls.
CloudTrail Lake	Managed, SQL-queryable event store. Retain events up to 7 years. No S3+Athena setup. Run cross-account queries.
EventBridge Integration	CloudTrail Management Events delivered to EventBridge in real-time — trigger automated responses to API calls.
CloudWatch Logs Integration	Stream trail to CloudWatch Logs — create metric filters and alarms on specific API patterns.
Data Events	Capture S3 object-level, Lambda invocation, DynamoDB row-level events — complete data access audit trail.

2.5 Real-World Use Cases

Scenario	How CloudTrail Helps
Security Incident Response	S3 bucket suddenly public. Athena query on CloudTrail logs: who called PutBucketAcl? When? From what IP? → Identify compromised credential.
PCI-DSS Compliance	Auditors require proof that only authorized admins accessed cardholder data systems. CloudTrail data events on S3 provide this log with integrity validation.
Root Account Monitoring	CloudWatch Logs metric filter detects root account usage → SNS alarm → immediate page to security team.
Insider Threat Detection	IAM user downloads 50,000 customer records from S3 at 2 AM. CloudTrail data events + Athena query reveals the exfiltration pattern.
AWS Config Change Tracking	Config uses CloudTrail as its data source to track who changed a security group rule.

Scenario	How CloudTrail Helps
Cost Attribution	Identify which IAM role launched 500 EC2 instances last night → charge back to responsible team.
Unusual Login Detection	CloudTrail Insights detects 10x spike in AssumeRole calls → GuardDuty flags → automated IAM policy suspension.

2.6 Practical Example — Detect Root Account Login via CloudWatch

```
# 1. Create CloudTrail trail delivering to CloudWatch Logs
aws cloudtrail create-trail \
  --name org-wide-trail \
  --s3-bucket-name my-cloudtrail-logs \
  --cloud-watch-logs-log-group-arn arn:aws:logs:us-east-1:123:log-group:CloudTrailLogs \
  --cloud-watch-logs-role-arn arn:aws:iam::123:role/CloudTrailCWRole \
  --is-multi-region-trail \
  --include-global-service-events

# 2. Create CloudWatch Metric Filter for root account usage
aws logs put-metric-filter \
  --log-group-name CloudTrailLogs \
  --filter-name RootAccountUsage \
  --filter-pattern '{ $.userIdentity.type = "Root" && $.userIdentity.invokedBy NOT EXISTS
                    && $.eventType != "AwsServiceEvent" }' \
  --metric-transformations \
    metricName=RootAccountUsageCount,metricNamespace=CloudTrailMetrics,metricValue=1

# 3. Create CloudWatch Alarm that fires on ANY root usage
aws cloudwatch put-metric-alarm \
  --alarm-name RootAccountUsageAlarm \
  --metric-name RootAccountUsageCount \
  --namespace CloudTrailMetrics \
  --statistic Sum \
  --period 300 \
  --threshold 1 \
  --comparison-operator GreaterThanOrEqualToThreshold \
  --evaluation-periods 1 \
  --alarm-actions arn:aws:sns:us-east-1:123:SecurityAlerts
```

■ *Note: Always enable a multi-region, organization-level trail. Attackers often operate in unused regions to avoid detection. Log file integrity validation is required for most compliance frameworks — enable it by default.*

2.7 Integrations & Comparison

Service	Integration
Amazon S3	Primary log destination. Best practice: enable S3 Object Lock (WORM) on CloudTrail bucket to prevent deletion.
Amazon Athena	Query CloudTrail logs in S3 with SQL. Athena has a built-in CloudTrail table creator in its console.
Amazon CloudWatch Logs	Stream events for real-time metric filters and alarms on specific API patterns.
Amazon EventBridge	CloudTrail management events appear on Default Event Bus in real-time — trigger Lambda or Step Functions.

Service	Integration		
Amazon GuardDuty	GuardDuty analyzes CloudTrail logs as one of its three threat intelligence feeds.		
AWS Security Hub	Security Hub ingests CloudTrail-based findings from GuardDuty, Config, and Inspector.		
CloudTrail Lake	Native SQL query interface over CloudTrail — eliminates the S3+Glue+Athena setup.		
Dimension	CloudTrail	VPC Flow Logs	AWS Config
What it captures	API calls (who did what)	Network traffic (IP flows)	Resource configuration state
Granularity	Per API call	Per network connection	Per config change
Use Case	Audit, security forensics	Network troubleshooting, anomaly	Compliance, config history
Real-time	Near real-time (15 min to S3)	Near real-time	Near real-time
Together	All three complement each other — CloudTrail: who, Flow Logs: where/how (network), Config: what changed		

Amazon CloudWatch

Unified Observability — Metrics, Logs, Alarms, Dashboards & Application Insights

3.1 Simple Introduction

What is Amazon CloudWatch?

Amazon CloudWatch is AWS's unified observability service — it collects, monitors, and visualizes operational data (metrics, logs, and traces) from virtually every AWS service and your own applications. CloudWatch lets you understand what is happening inside your infrastructure in real time, set alarms when things go wrong, automatically react to problems, and visualize operational health on dashboards. It is the operational nerve centre of every well-managed AWS environment.

CloudWatch covers three pillars of observability: **Metrics** (numbers over time — CPU utilization, request count, error rate), **Logs** (text output from applications and AWS services), and **Traces** (request flow across distributed services, via integration with AWS X-Ray).

■ **CloudWatch is like the vital signs monitor in an ICU.** Every patient (AWS resource) has sensors attached — heart rate (CPU), blood pressure (memory/disk), oxygen (network I/O). The monitor (CloudWatch) displays real-time graphs, sounds an alarm when readings exceed safe thresholds, keeps historical records for analysis, and automatically calls the doctor (Lambda/SNS) when a patient's condition deteriorates. Without it, you're flying blind.

3.2 Core Concepts & Terminology

Term	Definition
Metric	A time-ordered set of data points representing a measurable value. e.g., EC2 CPUUtilization in %. Identified by: Namespace, MetricName, Dimensions.
Namespace	Container for metrics — groups related metrics: AWS/EC2, AWS/Lambda, AWS/RDS, or your custom namespace.
Dimension	A key-value pair that identifies a metric within a namespace: InstanceId=i-12345, FunctionName=my-function.
Resolution	Standard: 1-minute granularity (default). High-Resolution: 1-second granularity (custom metrics, extra cost).
Alarm	Monitors a single metric and triggers action when it breaches a threshold for N consecutive periods.
Alarm States	OK (within threshold), ALARM (threshold breached), INSUFFICIENT_DATA (not enough data yet).
Composite Alarm	An alarm that combines multiple alarms using AND/OR logic — reduce alarm noise.
Anomaly Detection	ML model that learns a metric's normal behavior and alarms on statistical anomalies instead of fixed thresholds.
Log Group	Container for log streams. Defines retention period and access permissions. e.g., /aws/lambda/my-function.

Term	Definition
Log Stream	Sequence of log events from a single source within a Log Group.
Log Insights	Interactive SQL-like query language for analyzing log data across groups.
Metric Filter	Extracts metric values from log data (e.g., count ERROR occurrences → create ErrorCount metric).
Dashboard	Customizable visualization of metrics and logs — graphs, numbers, text, alarms — across multiple services.
CloudWatch Agent	Software installed on EC2/on-prem servers to push system-level metrics (memory, disk) and logs to CloudWatch.
CloudWatch Synthetics	Configurable canary scripts that continuously test your application endpoints (API, UI) from outside.
Contributor Insights	Identifies top contributors to performance problems in logs (e.g., top 10 IPs causing 4xx errors).
Container Insights	Metrics and logs for ECS, EKS, and Kubernetes — container, pod, and task level granularity.
Application Insights	Automatically detects common application patterns and configures monitoring for .NET, IIS, Java apps.

3.3 Architecture & Working

DATA SOURCES

AWS Services → automatic metrics (EC2, RDS, Lambda, ALB, DynamoDB...)

CloudWatch Agent → EC2/on-prem OS metrics (memory, disk) + application logs

SDK/API (PutMetricData) → custom application metrics from any code

Kinesis Firehose / Logs Subscription → stream logs into CloudWatch

↓

CLOUDWATCH DATA PLANE

Metrics Store – time-series data, retained 15 months (1-min granularity)

Logs Store – log groups with configurable retention (1 day – 10 years)

↓

PROCESSING LAYER

Alarms – evaluate metrics every period, change state, trigger actions

Metric Filters – scan logs, extract values, create derived metrics

Logs Insights – ad-hoc queries across log groups

Anomaly Detection – ML bands around expected metric values

↓

ACTIONS & OUTPUTS

SNS Notifications → email, SMS, PagerDuty, Slack

Auto Scaling → scale EC2/ECS/Lambda based on metric alarms

EC2 Actions → stop, start, reboot, recover EC2 instances

Systems Manager OpsItem → create incident ticket automatically

EventBridge → route alarm state changes to any target

Dashboards → visualize for ops teams

3.4 Major Features

Feature	Description	Use Case
CloudWatch Alarms	Monitor any metric. Single, composite, or anomaly-based. Actions: SNS, Auto Scaling, EC2 action.	Alert on high CPU, low disk, Lambda errors, ALB 5xx rate
CloudWatch Logs Insights	SQL-like query over log groups. Parse JSON/structured logs. Visualize query results.	Debug Lambda cold starts, trace specific user requests, count errors
Metric Math	Combine multiple metrics with math expressions: $\text{ErrorRate} = \text{Errors/Requests} \times 100$	Create derived metrics without custom code
Dashboards	Custom widgets: line graphs, stacked area, numbers, pie, text, alarms, log queries	Executive ops dashboards, service health pages
Anomaly Detection	ML learns baseline, creates dynamic threshold band. Alarm when metric goes outside band.	Detect unusual traffic spikes, latency degradation
Synthetics Canaries	Scripted browser tests (Puppeteer/Selenium) run on a schedule from multiple regions.	Detect broken user journeys before customers do
Contributor Insights	Top-N analysis on log data — who/what is generating the most load or errors.	Find top 10 IPs causing 429 errors, slowest Lambda functions
Container Insights	ECS/EKS metrics: CPU/memory per task, pod crash counts, restart rates.	Kubernetes and ECS operational visibility
CloudWatch Agent	Collect OS-level metrics (RAM, disk I/O, processes) and custom log files from EC2.	Memory utilization (not available from EC2 by default)
Logs Subscription Filter	Stream logs in real-time to Lambda, Kinesis, or OpenSearch for processing.	Real-time log analysis pipeline

3.5 Real-World Use Cases

Scenario	Details
Auto Scaling Trigger	ALB RequestCount metric > 1000/min for 2 consecutive periods → alarm → EC2 Auto Scaling Group adds 5 instances.
Lambda Error Rate Monitoring	Metric filter on /aws/lambda/payment-service logs counts ERROR lines → ErrorCount metric → alarm → PagerDuty incident.
Database Performance	RDS ReadLatency and WriteLatency metrics on dashboard. Alarm when P99 > 100ms → alert DBA team.
Canary for E-Commerce	Synthetics canary clicks through checkout flow every 5 minutes. If it fails → CloudWatch alarm → incident.
Log-Based Billing Alert	Metric filter scans billing logs, extracts spend amounts → composite alarm when daily spend > \$500.

Scenario	Details
Security Anomaly	CloudTrail logs delivered to CloudWatch → metric filter counts ConsoleLogin failures → composite alarm for brute-force.
Container Health	EKS pod restart count > 5 in 10 minutes via Container Insights → alarm → Slack alert to on-call engineer.

3.6 Practical Example — Custom Metric + Alarm + Auto Scaling

```
import boto3, time

cw = boto3.client('cloudwatch')

# 1. Publish custom metric from your application
cw.put_metric_data(
    Namespace='MyApp/Orders',
    MetricData=[{
        'MetricName': 'OrderProcessingTime',
        'Value': 245.0,          # milliseconds
        'Unit': 'Milliseconds',
        'Dimensions': [
            {'Name': 'Environment', 'Value': 'production'},
            {'Name': 'Service', 'Value': 'checkout'}
        ]
    }]
)

# 2. Create alarm: trigger if avg latency > 500ms for 3 consecutive 1-min periods
cw.put_metric_alarm(
    AlarmName='CheckoutLatencyHigh',
    MetricName='OrderProcessingTime',
    Namespace='MyApp/Orders',
    Dimensions=[{'Name': 'Environment', 'Value': 'production'}],
    Statistic='Average',
    Period=60,                # 1-minute evaluation period
    EvaluationPeriods=3,      # Alarm only after 3 consecutive breaches
    Threshold=500.0,
    ComparisonOperator='GreaterThanThreshold',
    AlarmActions=['arn:aws:sns:us-east-1:123:ops-alerts'], # page on-call
    OKActions    =['arn:aws:sns:us-east-1:123:ops-alerts'], # notify recovery
    TreatMissingData='notBreaching'
)

# 3. CloudWatch Logs Insights – find slow Lambda invocations
# Run this query on /aws/lambda/checkout-function:

fields @timestamp, @duration, @memorySize
| filter @duration > 2000          -- requests over 2 seconds
| sort @duration desc
| limit 20
| stats count(*) as SlowCount,
    avg(@duration) as AvgDuration,
    max(@duration) as MaxDuration
```

3.7 Integrations & Comparison

Service	Integration		
AWS Lambda	Automatic logs to /aws/lambda/. Metrics: Duration, Errors, Invocations, ConcurrentExecutions, Throttles.		
Amazon EC2	Automatic: CPU, Network, Disk. CloudWatch Agent needed for: Memory, Disk utilization, custom logs.		
Amazon RDS	Automatic: CPU, connections, IOPS, latency, FreeStorageSpace. Enhanced Monitoring: per-process OS metrics.		
Amazon ECS/EKS	Container Insights collects cluster, service, task, and pod level metrics automatically.		
AWS X-Ray	Trace requests through distributed services. CloudWatch ServiceLens integrates X-Ray traces with CW metrics.		
Amazon EventBridge	Alarm state changes (OK→ALARM) published as EventBridge events for complex routing.		
AWS Auto Scaling	Alarms trigger scaling policies for EC2 ASG, ECS services, DynamoDB, Lambda concurrency.		
Feature	CloudWatch	Grafana (AMG)	Prometheus
Data Sources	AWS-native only	Multi-source (CW, Prometheus, X-Ray, etc.)	Kubernetes/app metrics
Dashboard Quality	Good (basic)	Excellent (highly customizable)	Good
Alerting	Yes (alarms)	Yes (Grafana Alerting)	Yes (AlertManager)
Managed by AWS	Fully managed	Managed (MWAA-style)	Self-managed or via AMG
Cost	Pay per metric/alarm/log GB	Per active user per month	Open source
Best For	AWS-native monitoring	Multi-cloud, rich viz, existing Grafana users	Kubernetes workload metrics

AWS Config

Continuous Configuration Compliance — Track, Evaluate & Remediate Resource Configurations

4.1 Simple Introduction

What is AWS Config?

AWS Config is a service that continuously records the configuration state of your AWS resources, evaluates those configurations against desired rules, and provides a complete history of every configuration change. If an S3 bucket becomes public, if a security group opens port 22 to the internet, or if an EC2 instance launches without encryption — AWS Config detects it, records it, and can automatically remediate it. Config answers two critical questions: **What is the current configuration of my resources?** and **What was the configuration at any point in the past?**

■ **AWS Config is like a continuous home inspector who also keeps a photo album.** Imagine an inspector who visits your house every few minutes, photographs every room, and checks against a building code (Config Rules). If they find a fire hazard (open security group port), they flag it immediately. They also keep every photo ever taken — so if something broke last Tuesday, you can flip back to Tuesday's photos and see exactly what changed.

4.2 Core Concepts & Terminology

Term	Definition
Configuration Item (CI)	A snapshot of a resource's configuration at a specific point in time. Includes: resource type, ID, ARN, tags, relationships, and all configuration attributes.
Configuration History	The complete chronological record of all CIs for a resource — see every state it was ever in.
Configuration Recorder	The component that detects resource changes and records new Configuration Items. One recorder per region.
Configuration Snapshot	Point-in-time dump of all resource configurations in an account/region — delivered to S3 on demand or schedule.
Config Rule	An evaluation against a desired configuration state. Result: COMPLIANT or NON_COMPLIANT. Two types: AWS Managed and Custom.
AWS Managed Rule	Pre-built rules by AWS covering 200+ common compliance checks: s3-bucket-public-read-prohibited, ec2-instance-no-public-ip, encrypted-volumes, etc.
Custom Config Rule	Rules you write as Lambda functions — evaluate any resource configuration logic you define.
Conformance Pack	A collection of Config Rules and remediation actions packaged together. Deploy org-wide standards (CIS Benchmark, HIPAA, PCI-DSS packs).
Remediation Action	Automated response to a non-compliant finding: runs an SSM Automation document to fix the issue. Auto or manual trigger.
Resource Timeline	Visual history of a resource's configuration changes, relationships, and CloudTrail events over time.

Term	Definition
Aggregator	Collects Config data from multiple accounts and regions into a single view for compliance dashboards.
Conformance Pack Status	Dashboard showing pass/fail rates across all rules in a conformance pack for all resources.

4.3 Architecture & Working

RESOURCE CHANGES (Any AWS resource created/modified/deleted)

↓ AWS Config detects change via CloudTrail integration

CONFIGURATION RECORDER

Creates Configuration Item (JSON snapshot of resource state)

Stores CI in Configuration History

Delivers to S3 bucket (optionally SNS notification)

↓ Triggers rule evaluation

CONFIG RULES ENGINE

Rule Type 1: Configuration Change – evaluate when resource changes

Rule Type 2: Periodic – evaluate all resources every N hours

→ AWS Managed Rules: call AWS-managed Lambda with resource config

→ Custom Rules: call your Lambda function with resource config JSON

→ Result: COMPLIANT / NON_COMPLIANT / NOT_APPLICABLE

↓ Non-compliant finding

REMEDIATION

Manual: Config console shows violation → admin reviews → manual fix

Auto: Config triggers SSM Automation document → auto-fix resource

↓

AGGREGATOR (multi-account/region compliance dashboard)

Central account aggregates: 500 accounts × 20 regions → one dashboard

4.4 Major Features

Feature	Description
200+ Managed Rules	Pre-built checks for security, cost, and operational best practices. Enable in one click.
Custom Rules (Lambda)	Write any compliance logic in Python/Node.js — evaluate any configuration attribute.
Conformance Packs	Bundled rule sets for compliance standards: CIS AWS Benchmark, NIST 800-53, PCI-DSS, HIPAA, GDPR.
Automated Remediation	SSM Automation documents fix non-compliant resources automatically (e.g., encrypt EBS volume, remove public S3 ACL).
Resource Timeline	See every configuration state and relationship change for any resource over its entire life.

Feature	Description
Multi-Account Aggregator	Aggregate compliance across all accounts in an AWS Org. Compliance score per account/rule.
Advanced Query	SQL-based queries on current resource state across your entire organization: 'Find all unencrypted S3 buckets'.
Proactive Evaluation	Evaluate CloudFormation template resources against Config Rules BEFORE deployment (shift-left compliance).

4.5 Real-World Use Cases

Scenario	Config Rule Used	Remediation
S3 Bucket Made Public	s3-bucket-public-read-prohibited, s3-bucket-public-write-prohibited	Auto: SSM removes public ACL + disables public access block
Unencrypted EBS Volume	encrypted-volumes	Auto: Take snapshot → create encrypted copy → replace original
EC2 with Public IP	ec2-instance-no-public-ip	Alert: SNS → team reviews; terminate if unauthorized
Open SSH Port (0.0.0.0/0)	restricted-ssh	Auto: SSM removes the offending security group rule
MFA Not on Root	root-account-mfa-enabled (Periodic)	Alert: send to Security Hub + notify CISO via SNS
Outdated AMI	approved-amis-by-id	Flag instances running unapproved AMIs → replace in maintenance window
IAM Password Policy Weak	iam-password-policy	Alert: compliance dashboard flags account — IAM admin corrects

4.6 Practical Example — Custom Config Rule (Lambda)

```
import json

# Custom Config Rule: Ensure all EC2 instances have a 'CostCenter' tag
def lambda_handler(event, context):
    config = boto3.client('config')
    invoking_event = json.loads(event['invokingEvent'])

    # Get the EC2 instance configuration
    configuration_item = invoking_event['configurationItem']
    resource_type = configuration_item['resourceType']

    # Only evaluate EC2 instances
    if resource_type != 'AWS::EC2::Instance':
        return put_result(config, event, 'NOT_APPLICABLE', 'Not an EC2 instance')

    tags = configuration_item.get('tags', {})

    if 'CostCenter' in tags and tags['CostCenter'].strip():
        compliance = 'COMPLIANT'
        annotation = f"CostCenter tag present: {tags['CostCenter']}"
```

```

else:
    compliance = 'NON_COMPLIANT'
    annotation = 'Missing required CostCenter tag'

    return put_result(config, event, compliance, annotation)

def put_result(config_client, event, compliance_type, annotation):
    config_client.put_evaluations(
        Evaluations=[{
            'ComplianceResourceType': json.loads(event['invokingEvent'])
                                   ['configurationItem']['resourceType'],
            'ComplianceResourceId':   json.loads(event['invokingEvent'])
                                   ['configurationItem']['resourceId'],
            'ComplianceType': compliance_type,
            'Annotation': annotation,
            'OrderingTimestamp': json.loads(event['invokingEvent'])
                                   ['configurationItem']['configurationItemCaptureTime']
        }],
        ResultToken=event['resultToken']
    )

```

4.7 Comparison: Config vs CloudTrail vs Security Hub

Dimension	AWS Config	AWS CloudTrail	AWS Security Hub
Focus	Resource configuration state & compliance	API call history & audit	Aggregated security findings
Question Answered	What is/was configured?	Who did what?	What are my security risks?
Real-time	Near real-time (minutes)	Near real-time (15 min to S3)	Near real-time (aggregated)
Remediation	Yes (SSM Automation)	No (audit only)	No (references other services)
Compliance Reporting	Conformance Packs with scores	Not natively	CSPM findings + scores
Best Used For	Compliance automation, config history	Security forensics, audit	Security posture dashboard

Amazon Managed Grafana

Managed Visualization Platform — Rich Operational Dashboards Across Any Data Source

5.1 Simple Introduction

What is Amazon Managed Grafana?

Amazon Managed Grafana (AMG) is a fully managed service based on the open-source Grafana observability platform. Grafana is the industry-standard tool for creating rich, interactive operational dashboards that visualize metrics, logs, and traces from multiple data sources in a single pane of glass. AWS manages the Grafana server infrastructure, handles upgrades, provides high availability, and integrates natively with AWS authentication (IAM Identity Center / SSO) and AWS data sources — so you get the full power of Grafana without running servers.

Why Grafana Over CloudWatch Dashboards?

CloudWatch dashboards are good but limited to AWS data sources. Grafana connects to 50+ data sources simultaneously — CloudWatch, Prometheus, InfluxDB, PostgreSQL, MySQL, Elasticsearch, Jaeger, Zipkin, GitHub, Jira, and more. Grafana's visualization capabilities are far richer: heatmaps, candlestick charts, geo maps, node graph, state timelines, histogram panels, and hundreds of community plugins.

■ **Grafana is like a universal remote control for all your data.** Imagine having 10 different TVs (CloudWatch metrics, Prometheus from Kubernetes, your PostgreSQL database stats, IoT sensor data, business KPIs). Each TV has its own remote (separate dashboard). Grafana is the one universal remote that controls all TVs simultaneously, showing them all on one wall-sized screen — correlated by time.

5.2 Core Concepts & Terminology

Term	Definition
Workspace	A managed Grafana instance (server). You create one or more workspaces — each is an isolated Grafana environment with its own URL, users, and dashboards.
Data Source	A connection to a backend system providing data: CloudWatch, Prometheus, InfluxDB, PostgreSQL, MySQL, OpenSearch, Timestream, X-Ray, etc.
Panel	A single visualization block on a dashboard: time-series graph, stat, gauge, bar chart, table, heatmap, text, etc.
Dashboard	A collection of panels arranged on a canvas. Dashboards have time range controls, variables, and refresh intervals.
Query	Request to a data source for specific data: PromQL (Prometheus), CloudWatch Metric Math, SQL, Lucene (OpenSearch).
Variable	Dynamic filter that users can change to update all panels (e.g., select Environment: dev/staging/prod).
Alert	Grafana-native alerting — evaluate queries periodically, send notifications via Slack, PagerDuty, SNS, OpsGenie when conditions are met.
Folder	Organize dashboards into folders for teams or services.

Term	Definition
Plugin	Extend Grafana with additional data sources, panel types, and apps. 100+ community plugins available.
AWS IAM Identity Center	Single Sign-On for workspace access — employees log in with their corporate credentials, no separate Grafana users.
Loki Integration	Connect to Amazon Managed Service for Prometheus (AMP) and Grafana Loki for logs alongside metrics.

5.3 Architecture & Working

DATA SOURCES

Amazon CloudWatch – AWS service metrics, alarms, logs

Amazon Timestream – IoT time-series data

Amazon OpenSearch – log analytics and search data

AWS X-Ray – distributed traces

Amazon Managed Prometheus (AMP) – Kubernetes/app metrics via PromQL

External: PostgreSQL ■ MySQL ■ InfluxDB ■ Loki ■ Jaeger ■ GitHub

↓ Grafana queries each source independently

AMAZON MANAGED GRAFANA WORKSPACE (fully managed by AWS)

■ Grafana Server (HA, managed) – accessible via HTTPS URL ■

■ Authentication: AWS IAM Identity Center (SSO/SAML) ■

■ Permissions: AWS IAM for data source access ■

■ Plugin management: install from Grafana marketplace ■

↓ Users access via browser

GRAFANA UI

Dashboard designer → Panels → Queries → Visualizations

Alerting → Rules → Contact Points (Slack, PagerDuty, SNS, Email)

5.4 Major Features

Feature	Description
Managed & Scalable	AWS provisions, upgrades, patches, and scales Grafana. No EC2 or Kubernetes needed.
Native AWS Data Sources	Pre-configured connectors for CloudWatch, X-Ray, OpenSearch, Timestream, IoT SiteWise, AMP, Redshift.
IAM Identity Center Auth	Enterprise SSO — users log in with their corporate identity. No separate Grafana user management.
AWS Permission Boundaries	IAM roles control exactly which CloudWatch namespaces/log groups each workspace can read.
Grafana Alerting	Rich alerting rules (PromQL, CloudWatch math), notification channels, silences, and alert routing.

Feature	Description
Multi-Source Dashboards	Single dashboard panel queries CloudWatch AND Prometheus AND RDS — correlate data from any source.
50+ Visualization Types	Time-series, stat, gauge, heatmap, geo map, candlestick, state timeline, histogram, and more.
Community Plugins	Access hundreds of open-source Grafana plugins for additional data sources and visualizations.
Dashboard Versioning	Track dashboard changes — revert to previous versions if a dashboard is accidentally broken.
High Availability	Multi-AZ deployment managed by AWS — 99.9% SLA for workspaces.

5.5 Real-World Use Cases

Industry	Use Case
DevOps / Platform Engineering	Single Grafana dashboard: Kubernetes pod health (Prometheus) + Lambda errors (CloudWatch) + CDN latency (CloudWatch) — one pane of glass.
IoT / Manufacturing	Amazon Timestream stores sensor data from 10,000 machines. Grafana displays real-time temperature, vibration, and OEE on factory floor screens.
Fintech	Trade execution dashboard: order latency (Prometheus) + error rate (CloudWatch) + market data lag (InfluxDB) with 1-second refresh.
SaaS Platform	Customer-facing status page embedded Grafana dashboard showing API uptime, P99 latency, and incident history.
Healthcare IT	Hospital operations: bed utilization (PostgreSQL) + lab system latency (Prometheus) + EMR API errors (CloudWatch) in one dashboard.
E-Commerce	Black Friday dashboard: orders per minute (CloudWatch) + cart abandonment rate (custom metric) + payment gateway latency (Datadog via plugin).

5.6 Practical Example — Create Workspace & Add CloudWatch Source

```
# Create AMG Workspace via AWS CLI
aws grafana create-workspace \
  --workspace-name 'ops-dashboards' \
  --account-access-type CURRENT_ACCOUNT \
  --authentication-providers AWS_SSO \
  --permission-type SERVICE_MANAGED \
  --workspace-data-sources CLOUDWATCH XRAY PROMETHEUS TIMESTREAM

# Grant IAM Identity Center group 'DevOps' as Grafana Admin
aws grafana create-workspace-role-association \
  --workspace-id ws-XXXXXXXXXX \
  --role Admin \
  --group-ids 'devops-group-id-from-identity-center'

# The workspace URL will be: https://ws-XXXXXXXXXX.grafana-workspace.us-east-1.amazonaws.com
# Users visit this URL, authenticate via SSO, and access Grafana immediately.

# Grafana dashboard JSON snippet – CloudWatch metric panel:
# {
```

```
# "type": "timeseries",
# "targets": [{
#   "datasource": {"type": "cloudwatch"},
#   "dimensions": {"FunctionName": ["$function_name"]}, ← uses variable
#   "expression": "",
#   "id": "e1",
#   "matchExact": true,
#   "metricEditorMode": 0,
#   "metricName": "Errors",
#   "namespace": "AWS/Lambda",
#   "period": "60",
#   "queryMode": "Metrics",
#   "statistic": "Sum"
# }],
# "title": "Lambda Errors per Minute"
# }
```

5.7 Comparison: AMG vs CloudWatch Dashboards vs QuickSight

Dimension	Amazon Managed Grafana	CloudWatch Dashboards	Amazon QuickSight
Primary Use	Operational / infrastructure monitoring	AWS infrastructure monitoring	Business intelligence / analytics
Data Sources	50+ (AWS + any external)	AWS only	AWS + SaaS + databases
Viz Richness	Excellent (50+ panel types)	Good (basic)	Excellent (30+ chart types)
Real-Time	Yes (1-second refresh)	Yes (1-minute+)	Near real-time (SPICE refresh)
Target Users	DevOps, SRE, Platform engineers	Cloud operations teams	Business analysts, executives
Authentication	IAM Identity Center (SSO/SAML)	AWS Console IAM	AWS IAM + QuickSight users
Cost	Per active user/month (~\$9)	Per dashboard + API calls	Per author + per session
Best For	Multi-source ops dashboards, K8s monitoring	Quick AWS-native monitoring	Business KPI dashboards, embedded analytics

AWS Systems Manager

Unified Operations Hub — Manage, Configure & Automate Infrastructure at Scale

6.1 Simple Introduction

What is AWS Systems Manager?

AWS Systems Manager (SSM) is a unified management service that provides visibility and control over your entire AWS infrastructure — EC2 instances, on-premises servers, and even VMs in other clouds. SSM is not a single feature but a collection of capabilities covering operational tasks: securely connecting to servers without SSH keys, storing application configuration securely, automating patching, running commands across thousands of servers simultaneously, and maintaining operational health. It is the 'ops swiss army knife' of AWS.

Key Sub-Services at a Glance

Capability	One-Line Description
Session Manager	Browser/CLI-based shell access to EC2 and on-prem servers — no SSH, no bastion host, no inbound ports needed.
Parameter Store	Secure, hierarchical storage for configuration data and secrets (connection strings, API keys, feature flags).
Secrets Manager	(Separate service) Fully managed secrets rotation — builds on Parameter Store concepts.
Patch Manager	Automate OS and application patching across fleets of EC2/on-prem servers on a schedule.
Run Command	Execute any shell/PowerShell command across thousands of instances simultaneously — no SSH needed.
Automation	Run multi-step automation playbooks (called Runbooks) using pre-built or custom SSM Documents.
Inventory	Collect and query software inventory, installed packages, running services from all managed instances.
State Manager	Define and enforce desired configuration state — ensure instances always have specific software or settings.
OpsCenter	Centralized operational issues (OpsItems) workspace — aggregate alarms, findings, and events into tickets.
Maintenance Windows	Define maintenance windows to run disruptive operations (patching, restarts) during off-hours.
Distributor	Deploy and update software packages across your instance fleet.
AppConfig	Safely deploy application configuration changes with rollback — separate from code deployments.

■ **SSM is like a building management system for your data center.** Instead of physically walking to each server room (SSH to each instance), the building manager (SSM) has a master control panel — press a button to unlock any door (Session Manager), check what software is installed in each room (Inventory), push a configuration change to all rooms simultaneously (Run Command), schedule maintenance across all floors (Maintenance Windows + Patch Manager), and ensure every room meets

the fire code (State Manager).

6.2 Core Concepts & Terminology

Term	Definition
SSM Agent	Lightweight agent installed on EC2 instances (pre-installed on Amazon Linux 2, Windows Server AMIs). Communicates with SSM service via HTTPS — no inbound ports needed.
Managed Instance	Any server (EC2, on-prem, VM) with SSM Agent installed and registered with SSM. Has an instance ID visible in SSM console.
SSM Document	JSON/YAML definition of actions to perform (like a script with steps). Types: Command, Session, Automation, Policy.
Run Command	Execute a Command Document across target instances. Specify targets by tag, instance ID, or resource group.
Session Manager Session	An encrypted, audited interactive shell to an instance. No SSH key, no bastion, no port 22 needed.
Parameter Store Hierarchy	/dept/app/env/key — hierarchical naming for parameters. Access controlled by IAM on path prefix.
SecureString Parameter	Parameter encrypted with KMS — stores passwords, API keys, connection strings securely.
Patch Baseline	Defines which patches are auto-approved for installation: critical patches, severity levels, delay.
Patch Group	EC2 instances tagged with Patch Group= — different groups can have different patch baselines.
Automation Runbook	Multi-step SSM Document for automation playbooks: stop instance → create snapshot → start instance.
AppConfig Deployment Strategy	How fast an AppConfig change rolls out: instant, linear 10%/min, exponential, or bake time.
Resource Group	Collection of AWS resources sharing tags — use as target for Run Command or Automation.

6.3 Architecture — Session Manager Deep Dive

TRADITIONAL SSH ACCESS (problematic)

User → Internet → Port 22 open → Bastion Host → Private EC2

Problems: key management, bastion maintenance, open inbound ports, no audit log

SESSION MANAGER (secure, no bastion needed)

User's Browser/CLI → IAM Authentication → SSM Service Endpoint (HTTPS 443)

↓ WebSocket tunnel

SSM Agent on EC2 ← polls SSM endpoint (outbound HTTPS only)

Interactive shell session established through this tunnel

Session commands logged to: S3 bucket + CloudWatch Logs

All session activity auditable via CloudTrail

PARAMETER STORE ARCHITECTURE

Application Code → GetParameter API → SSM (checks IAM) → returns value
Values encrypted at rest with KMS (SecureString type)
Lambda / ECS / EC2 apps retrieve config at runtime (no env var secrets)
Cross-account access via resource policies

6.4 Major Features

Capability	Key Feature	Business Value
Session Manager	Browser shell. No port 22. Full audit log. IAM-based access.	Eliminate bastion hosts and SSH key sprawl
Parameter Store	Hierarchical, encrypted config store. Versioning. CloudFormation/Lambda native.	Centralize and secure all application configuration
Patch Manager	Automated patching schedule. Patch baselines. Compliance reporting.	Reduce vulnerability exposure without manual work
Run Command	Execute on 1–10,000 instances by tag. Rate control. Output to S3/CW.	Mass operations: deploy agents, collect logs, restart services
Automation	Multi-step runbooks. Human approval gates. EventBridge triggers.	Self-healing infrastructure; standardized ops procedures
Inventory	Software, services, Windows registry, network config from all instances.	Know what's running everywhere — security and compliance
AppConfig	Safely deploy config changes with rollback on CloudWatch alarms.	Feature flags, runtime config — without redeployment
OpsCenter	Aggregate OpsItems from CloudWatch Alarms, Config, GuardDuty. Runbook execution.	Single operations console for incident management
Maintenance Windows	Define time windows for disruptive tasks with rate and error controls.	Patch 10,000 servers during defined off-hours windows

6.5 Real-World Use Cases

Scenario	SSM Capability Used
No-SSH Server Access	Developer needs to debug a production EC2. Session Manager opens browser shell — no key pair, no security group change, full audit log.
Secure Config in Lambda	Lambda function retrieves DB password: <code>ssm.get_parameter(Name='/prod/db/password', WithDecryption=True)</code> . No hard-coded secrets.
Patch 5,000 Linux Servers	Patch Manager: baseline = CriticalSecurity patches. Maintenance Window: every Sunday 2–5 AM. CloudWatch dashboard shows patch compliance %.

Dimension	AWS Systems Manager	Ansible (open source)	Chef / Puppet
Agent Required	SSM Agent (pre-installed on AWS AMIs)	Agentless (SSH)	Agent required (complex)
Management	Fully managed by AWS	Self-managed control plane	Self-managed server required
Secrets Storage	Parameter Store (KMS encrypted, native)	Ansible Vault (manual)	Chef Vault / Puppet Hiera
AWS Integration	Deep native (IAM, CloudWatch, CloudTrail)	Via AWS modules	Via integrations
Cost	Included with AWS (Parameter Store pricing for higher tiers)	Open source (infra cost)	Expensive (enterprise licensing)
Session Access	Session Manager (no SSH, browser-based)	Requires SSH	Requires SSH
Best For	AWS workloads, AWS-native ops	Multi-cloud, agentless simplicity	Large, mature DevOps teams

AWS Well-Architected Tool

Evaluate & Improve Your Cloud Architecture Against AWS Best Practices

7.1 Simple Introduction

What is the AWS Well-Architected Tool?

The AWS Well-Architected Tool is a free service that helps you review the state of your workloads against AWS best practices across six architectural pillars. You answer a series of questions about your workload's design decisions, and the tool generates a report highlighting high-risk issues (HRIs), medium-risk issues (MRIs), and improvement recommendations — each linked to detailed implementation guidance. It is based on the **AWS Well-Architected Framework**, developed from reviewing thousands of customer architectures.

What is the AWS Well-Architected Framework?

The AWS Well-Architected Framework is a set of architectural best practices organized around six pillars, developed by AWS Solutions Architects over 10+ years of reviewing real customer workloads. Each pillar contains design principles, questions you should ask about your workload, and best practice guidance. The Framework has been used to review over 500,000 workloads worldwide.

■ **The Well-Architected Tool is like an MOT (vehicle inspection) for your cloud workload.** Just as a car inspection checks brakes, tyres, lights, and emissions against safety standards, the Well-Architected Tool checks your security, reliability, performance, cost, and sustainability against AWS best practices. It doesn't fix the car for you — it gives you a detailed report of what needs fixing and how to fix it, prioritized by risk level.

7.2 The Six Pillars of the Well-Architected Framework

Pillar	Definition	Key Design Principles
Operational Excellence	Run and monitor systems to deliver business value and continually improve processes and procedures	Perform operations as code; make frequent small changes; anticipate failure; learn from failures
Security	Protect information, systems, and assets while delivering business value through risk assessments and mitigation strategies	Implement strong identity; enable traceability; apply security at all layers; automate security best practices; protect data in transit and at rest
Reliability	Ensure a workload performs its intended function correctly and consistently when expected to	Automatically recover from failure; test recovery procedures; scale horizontally; stop guessing capacity; manage change through automation
Performance Efficiency	Use computing resources efficiently to meet system requirements and maintain efficiency as demand changes and technologies evolve	Democratize advanced technologies; go global in minutes; use serverless; experiment more often; consider mechanical sympathy
Cost Optimization	Avoid unnecessary costs — understand spending, control fund allocation, select right resources	Implement cloud financial management; adopt consumption model; measure overall efficiency; stop spending on undifferentiated heavy lifting; analyze and attribute expenditure

Pillar	Definition	Key Design Principles
Sustainability	Minimize environmental impacts of running cloud workloads	Understand impact; establish sustainability goals; maximize utilization; use managed services; use efficient hardware and software

7.3 Architecture & Working

STEP 1: Define Workload

Name your workload, describe it, choose industry and environment (prod/pre-prod)

↓

STEP 2: Answer Pillar Questions

Each pillar has 10–15 questions with multiple-choice answers:

e.g., OPS 1: 'How do you determine what your priorities are?'

e.g., SEC 2: 'How do you manage identities for people and machines?'

Answer: select all best practices you have implemented

↓

STEP 3: View Risk Report

Tool generates: HRIs (High Risk Issues) + MRIs (Medium Risk Issues)

Each risk linked to: WHY it's a risk + HOW to remediate + AWS docs link

↓

STEP 4: Create Improvement Plan

Prioritize HRIs → assign owners → set milestones

↓

STEP 5: Milestone & Tracking

Save milestones to track improvement over time

Re-review workload after making improvements → measure risk reduction

↓

STEP 6: Generate Report

PDF report for leadership: risk summary, pillar scores, improvement plan

Core Tool Components

Component	Description
Workload	The primary unit of review — represents a product, application, or service you're evaluating.
Lens	A set of pillar questions tailored for a specific domain. AWS provides: Foundational Technical Review (FTR), Serverless, SaaS, Analytics, IoT, Machine Learning, and more. Custom lenses also possible.
Milestone	A snapshot of a workload's review at a specific point in time. Track progress by comparing milestones.
High Risk Issue (HRI)	A best practice that's not being followed that poses the highest risk to the workload. Must be addressed.
Medium Risk Issue (MRI)	A best practice gap with moderate risk — address after HRIs.
Improvement Plan	Actionable list of HRIs/MRIs with AWS documentation links and suggested remediation steps.

Component	Description
Profile	Define the workload's business context (criticality, data sensitivity, revenue impact) to prioritize questions.
Share	Share a workload review with other AWS accounts — enable trusted advisors or AWS Partner reviews.

7.4 Major Features

Feature	Description
Free Service	The Well-Architected Tool itself is completely free. You only pay for AWS resources you use.
Custom Lenses	Create your own lens with custom questions for internal governance, industry-specific requirements, or partner assessments.
Multiple Lenses per Workload	Apply the base Framework + Serverless Lens + SaaS Lens simultaneously for comprehensive coverage.
Trusted Advisor Integration	The Tool complements AWS Trusted Advisor — Trusted Advisor checks real-time resource configs; WAT reviews design decisions.
AWS Partner Reviews	Share workload with AWS Partner Network (APN) Partners for professional Well-Architected reviews.
Milestone Tracking	Review same workload quarterly — track how many HRIs were resolved. Demonstrate improvement to leadership.
API Access	Automate workload creation, answer ingestion, and report generation via the Well-Architected Tool API.
Profiles	Custom question weightings based on workload business context — focus reviews on what matters most for your workload type.

7.5 Real-World Use Cases

Scenario	Details
Pre-Launch Review	Before launching a new product, run a Well-Architected review. Find that backup/recovery (REL 9) is not implemented → add automated backups before go-live.
Post-Incident Review	After a production outage, use REL pillar questions to identify missing design patterns (circuit breakers, health checks) that contributed to the incident.
Cost Reduction Program	Finance asks for 20% cloud cost reduction. COST pillar review identifies: oversized instances, unattached EBS volumes, expensive data transfer → \$50K/month savings found.
Compliance Readiness	Pre-audit: use Security pillar + FedRAMP lens to systematically identify gaps before external compliance audit.
Partner Engagement	AWS Partner reviews 50 customer workloads per quarter using the Tool. Generates reports, proposes remediation projects.
Acquisition Due Diligence	Company acquiring a startup runs Well-Architected review on their AWS workloads to assess technical debt and cloud risk before acquisition close.

Scenario	Details
Internal Governance	Platform team creates a Custom Lens encoding company-specific standards. All product teams must pass Custom Lens review before production deployment.

7.6 Practical Example — API Automation

```
import boto3

wa = boto3.client('wellarchitected')

# 1. Create a workload
workload = wa.create_workload(
    WorkloadName='payment-processing-api',
    Description='Core payment processing microservice',
    Environment='PRODUCTION',
    AwsRegions=['us-east-1', 'eu-west-1'],
    Lenses=['wellarchitected', 'serverless'],    # Apply 2 lenses
    ReviewOwner='platform-team@company.com',
    IndustryType='Finance',
    Industry='Banking'
)
workload_id = workload['WorkloadId']

# 2. List questions for Security pillar
questions = wa.list_answers(
    WorkloadId=workload_id,
    LensAlias='wellarchitected',
    PillarId='security'
)

# 3. Update an answer (mark best practices as implemented)
wa.update_answer(
    WorkloadId=workload_id,
    LensAlias='wellarchitected',
    QuestionId='sec_identities',
    SelectedChoices=[
        'sec_identities_identity_provider',    # Using IdP for humans
        'sec_identities_enforce_mechanisms',  # Enforcing MFA
        'sec_identities_service_workload_auth', # IAM roles for services
    ],
    Notes='All services use IAM roles. MFA enforced via SCP. Okta as IdP.'
)

# 4. Get risk summary
summary = wa.get_lens_review(
    WorkloadId=workload_id,
    LensAlias='wellarchitected'
)
risk_counts = summary['LensReview']['RiskCounts']
print(f"High Risk: {risk_counts.get('HIGH', 0)}")
print(f"Medium Risk: {risk_counts.get('MEDIUM', 0)}")
```

7.7 Comparison: Well-Architected Tool vs Trusted Advisor vs Security Hub

Dimension	Well-Architected Tool	AWS Trusted Advisor	AWS Security Hub
Focus	Architectural design quality review	Real-time resource-level checks	Security findings aggregation
Input	Human answers to design questions	Actual AWS resource configurations	Automated findings from services
Output	Risk report + improvement plan	Real-time recommendations	Security score + finding list
Frequency	Periodic (quarterly reviews)	Continuous (real-time)	Continuous (real-time)
Automation	Low (human-driven review)	High (fully automated checks)	High (fully automated)
Coverage	Architectural decisions + design	~200 specific resource checks	Security misconfigs + vulnerabilities
Cost	Free	Free (basic); Business/Enterprise plans for full	Per finding ingested
Best For	Architecture design review, pre-launch, post-incident	Day-to-day resource hygiene	Security posture management

Master Comparison: All 7 AWS Management & Governance Services

At-a-Glance Summary

Service	Type	Core Strength	Cost Model	Best For
CloudFormation	IaC	Declarative infrastructure provisioning	Free (pay for resources)	Repeatable, versioned infra deployments
CloudTrail	Audit Logging	Complete API call history + tampering detection	Per event recorded	Security forensics, compliance, audit
CloudWatch	Observability	Metrics + Logs + Alarms in one platform	Per metric/log GB/alarm	Infrastructure monitoring, alerting, scaling
AWS Config	Config Compliance	Continuous compliance evaluation + history	Per config item recorded	Config drift detection, compliance automation
Managed Grafana	Visualization	Multi-source operational dashboards	Per active user/month	Cross-source observability, IoT, K8s
Systems Manager	Ops Hub	Unified server management without SSH	Various per capability	Secure access, patching, config, automation
Well-Architected Tool	Architecture Review	Structured workload quality assessment	Free	Pre-launch review, post-incident, governance

How These Services Work Together — Governance Reference Architecture

PROVISION

AWS CloudFormation / CDK → deploy all infrastructure as code, version-controlled
 cfn-guard → validate templates against policy rules before deployment

↓

DETECT CHANGE & AUDIT

AWS CloudTrail → record every API call (who, what, where, when)
 AWS Config → record every configuration state change

↓ Config non-compliant finding

EVALUATE COMPLIANCE

AWS Config Rules / Conformance Packs → evaluate resources against policies
 Config Aggregator → compliance dashboard across 500 accounts

↓ Auto-remediate

REMEDiate & OPERATE

SSM Automation → run remediation runbooks (fix non-compliant resources)
 SSM Patch Manager → patch all EC2 instances during maintenance windows
 SSM Session Manager → secure access for operations without SSH
 SSM Parameter Store → secure config for all applications

↓

OBSERVE & ALERT

CloudWatch Metrics + Logs → unified monitoring of all resources
CloudWatch Alarms → SNS → PagerDuty / Slack / auto-remediation
Amazon Managed Grafana → rich multi-source dashboards for ops teams
↓

REVIEW & IMPROVE

Well-Architected Tool → quarterly workload reviews → improvement plan
Custom Lenses → enforce org-specific standards across all product teams

Quick Decision Guide

Q: I want to provision 50 AWS resources reliably and repeatably? → **AWS**

CloudFormation

Q: Someone deleted a resource – who did it and when? → **AWS CloudTrail + Athena**

Q: I need real-time CPU/memory alarms and auto-scaling? → **Amazon CloudWatch**

Q: I need to ensure S3 buckets are never public, auto-fix if they are? → **AWS Config + SSM Remediation**

Q: I want Grafana dashboards with CloudWatch + Prometheus data? → **Amazon Managed Grafana**

Q: I need to patch 2,000 servers without touching them individually? → **SSM Patch Manager**

Q: I want to SSH into an EC2 instance without opening port 22? → **SSM Session Manager**

Q: I want to store DB passwords securely for my Lambda functions? → **SSM Parameter Store**

Q: I want to review whether my architecture follows AWS best practices? →

Well-Architected Tool

Q: I want to deploy my app config independently of code deployments? → **SSM AppConfig**

End of AWS Management & Governance Complete Guide

All 7 services explained with beginner-friendly depth, real-world use cases, architecture flows, comparison tables, and hands-on code examples.